These two sets of graphs are recursively checked at lines #4-#6 and #7-#9 respectively; since the algorithm is recursive, we need a base case: we could use the empty graph, trivially, but since we know that all graphs with 4 or less vertices are planar, we can stop our recursion earlier (see line #1) and save computational resources.

The only thing remaining is checking if current input is a non-planar graph: normally we would just use another base case (actually, 2), at line #3, where we check if recursion brought us either K_5 or $K_{3,3}$ (for Kuratowski's theorem, we know that this means non-planar). In this case, though, we added another check at line #2, using Euler's inequalities to our advantage: as we have seen in section 15.2.2, if the graph we are examining has too many edges for its vertices, it must be non-planar.

To see how the utility methods performing these checks work, let's take a look at listings 15.3, 15.4 and 15.5.

Listing 15.3 Planarity Testing utility methods: Euler's invariants

```
function
violatesEulerConstraints(graph)                                    #1
  (n,m) ← (|graph.vertices|,
|graph.edges|)                              #2
  if m > 3 * n - 6 then
                  #3
    return true
  if not hasCycleOfLength3(graph) and m > 2 * n - 4 then
         #4
    return true
  return false
```